

Day1 종합 실습 · 데이터 수집 미니 파이프라인

3개 공개 API 비동기 동시 수집 → Pydantic v2 검증 → CSV·Parquet 저장·성능 비교

광주캠퍼스 4반 권정

과정 SKALA 2) AI의 서비스화 — 데이터분석 및 AIOps

제출일 2026-07-20

환경 Python 3.11 · venv · httpx 0.28 · pydantic 2.13 · pandas 3.0 · pyarrow 25.0

1 프로젝트 개요 & 구조

3개 공개 API를 `asyncio.gather()` 로 동시에 수집하고, 응답에서 필요한 필드를 추출해 **Pydantic v2**로 타입·범위를 검증한 뒤, 검증 통과 데이터를 **CSV·Parquet** 두 형식으로 저장하며 읽기/쓰기 성능을 비교한다. 수집·검증·저장을 독립 모듈로 분리해 테스트·재사용이 용이하도록 설계했다. 가상환경에서 사용한 패키지와 버전은 `requirements.txt` 로 관리해 동일한 실행 환경을 재현할 수 있도록 했다.

모듈	역할
<code>models.py</code>	Pydantic v2 스키마 (타입·범위 검증) + <code>from_api()</code> 필드 추출
<code>collect.py</code>	httpx AsyncClient + <code>asyncio.gather</code> 동시 수집, 일시적 오류 백오프 재시도
<code>storage.py</code>	CSV/Parquet 저장 + 읽기/쓰기 시간·크기 측정·비교
<code>main.py</code>	수집 → 검증 → 저장 → 비교 오케스트레이션
<code>test_models.py</code>	pytest: 스키마·파이프라인·재시도 검증 (8건)
<code>requirements.txt</code>	httpx·pydantic·pandas·pyarrow·pytest·ruff 등 실행 의존성 및 버전 관리(<code>pip freeze</code> 고정)

수집 API: Open-Meteo(서울 3일 기온·강수확률) · Countries.dev(한국 국가정보) · ip-api(IP 지역정보)

2 실행 결과

python practice_day1/main.py

3개 API 동시 수집 → 검증 통과 → 3종 데이터 각각 CSV·Parquet 저장·성능 비교까지 정상 완료.

● ① 파이프라인 실행 — python practice_day1/main.py

```
(.venv) jung@Kwonui-MacBookPro data-project % python practice_day1/main.py
[1] 비동기 수집 시작 (3개 API 동시 호출)
수집 성공: ['weather', 'country', 'ip']

[2] 스키마 검증
[OK] weather 검증 통과
[OK] country 검증 통과
[OK] ip 검증 통과

[3] CSV·Parquet 저장 및 성능 비교

[weather] 저장·성능 비교 - 72행 (평균)
  형식      쓰기 (ms)    읽기 (ms)    크기 (KB)
  csv        0.242        0.187        1.81
  parquet    0.217        0.305        3.36

[country] 저장·성능 비교 - 1행 (평균)
  형식      쓰기 (ms)    읽기 (ms)    크기 (KB)
  csv        0.162        0.176        0.12
  parquet    0.323        0.424        4.27

[ip] 저장·성능 비교 - 1행 (평균)
  형식      쓰기 (ms)    읽기 (ms)    크기 (KB)
  csv        0.165        0.200        0.14
  parquet    0.456        0.473        5.21

[OK] 파이프라인 완료 → 결과: /Users/jung/data-project/practice_day1/output
```

→ 3개 API 동시 수집 성공 · 3종 모두 검증 통과 · 각 CSV·Parquet 저장과 성능 비교까지 정상 종료.

● 산출물 확인 — CSV 3종 (weather · country · ip)

```
(.venv) jung@Kwonui-MacBookPro data-project % head -3 practice_day1/output/weather.csv
time,temperature_2m,precipitation_probability
2026-07-20T00:00,23.2,6
2026-07-20T01:00,22.9,15
(.venv) jung@Kwonui-MacBookPro data-project % cat practice_day1/output/country.csv
name,capital,region,subregion,population,area,alpha3_code
Korea (Republic of),Seoul,Asia,Eastern Asia,51780579,100210.0,KOR
(.venv) jung@Kwonui-MacBookPro data-project % cat practice_day1/output/ip.csv
query,country,country_code,city,region_name,lat,lon,isp,timezone
8.8.8.8,United States,US,Ashburn,Virginia,39.03,-77.5,Google LLC,America/New_York
(.venv) jung@Kwonui-MacBookPro data-project %
```

→ weather 72행 + country·ip 각 1행. 컬럼 헤더와 값이 정상 저장됨을 확인.

3 테스트 결과

pytest · 8 passed

● ② 단위·통합·재시도 테스트 — pytest practice_day1 -v

```
(.venv) jung@Kwonui-MacBookPro data-project % pytest practice_day1 -v
===== test session starts =====
platform darwin -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /Users/jung/data-project/.venv/bin/python3.11
cachedir: .pytest_cache
rootdir: /Users/jung/data-project
plugins: anyio-4.14.2
collected 8 items

practice_day1/test_models.py::test_weather_from_api_ok PASSED [ 12%]
practice_day1/test_models.py::test_country_from_api_ok PASSED [ 25%]
practice_day1/test_models.py::test_ip_from_api_ok PASSED [ 37%]
practice_day1/test_models.py::test_precipitation_out_of_range PASSED [ 50%]
practice_day1/test_models.py::test_country_population_must_be_positive PASSED [ 62%]
practice_day1/test_models.py::test_ip_status_failure_raises PASSED [ 75%]
practice_day1/test_models.py::test_validate_all_handles_bad_record PASSED [ 87%]
practice_day1/test_models.py::test_fetch_json_retries_transient_error PASSED [100%]

===== 8 passed in 0.40s =====
(.venv) jung@Kwonui-MacBookPro data-project %
```

→ 모델 검증·파이프라인 예외 처리·수집 재시도까지 8개 테스트 전부 PASSED.

4 코드 스타일 검사

ruff · 0 error

● ③ 스타일 검사 — ruff check practice_day1

```
(.venv) jung@Kwonui-MacBookPro data-project % ruff check practice_day1
All checks passed!
(.venv) jung@Kwonui-MacBookPro data-project %
```

→ 스타일 규칙 위반 0건 (All checks passed).

5 Git 커밋 이력

git log

● ④ 커밋 이력 — git log --oneline -10

```
(.venv) jung@Kwonui-MacBookPro data-project % git log --oneline -10
2f1e179 (HEAD -> day1/collect-pipeline, origin/day1/collect-pipeline) docs: day1 파이프라인 README 추가
7db505d test: 재시도 테스트 추가, validate_all 반환 타입 명시
79be75a feat: API 수집 일시적 오류 백오프 재시도 추가
90e94d2 test: validate_all 파이프라인 예외 처리 테스트 추가
b5f3701 feat: 3종 데이터 각각 CSV, Parquet 저장, 성능 비교
392fbc0 feat: 검증 통과 데이터 3종 모두 CSV, Parquet 저장 (weather/country/ip)
70f15a9 test: day1 스키마 검증 pytest 6건 추가
462a6cf feat: day1 수집→검증→저장 파이프라인 오케스트레이션 (main)
990fd4e feat: day1 CSV/Parquet 저장, 읽기쓰기 성능 비교 (storage)
125ecbf feat: day1 Pydantic 스키마(models), 비동기 수집(collect) 구현
```

→ 환경 구성부터 기능·테스트·문서화까지 의미 단위로 커밋을 분리.

6 성능 비교 분석 — CSV vs Parquet

데이터	행	CSV 쓰기/읽기 (ms)	Parquet 쓰기/읽기 (ms)	CSV / Parquet 크기
weather	72	0.242 / 0.187	0.217 / 0.305	1.81 / 3.36 KB
country	1	0.162 / 0.176	0.323 / 0.424	0.12 / 4.27 KB
ip	1	0.165 / 0.200	0.456 / 0.473	0.14 / 5.21 KB

핵심 관찰: 데이터가 작을수록 Parquet의 스키마·푸터 등 고정 메타데이터 오버헤드가 지배적이다. 1행 데이터에서 Parquet 파일이 CSV보다 **30배 이상** 크며, 72행 weather에서는 크기 격차가 1.9배로 줄어든다.

해석 및 결론

소량 데이터에서는 파일 크기와 읽기 성능 측면에서 CSV가 전반적으로 유리했다. weather 데이터의 쓰기에서는 Parquet가 소폭 빨랐지만(0.217 vs 0.242 ms) 차이가 매우 작았으며, country-ip 데이터에서는 CSV가 읽기·쓰기 모두 빨랐다. 현재 데이터 규모에서는 Parquet의 고정 메타데이터 오버헤드가 압축·컬럼형 저장의 장점보다 크게 나타났다.

- ▶ **대용량 데이터:** 행이 수만~수백만이 되면 Parquet의 컬럼형 압축·타입 보존·컬럼 선택 조회(column/predicate pruning)가 압도적으로 유리해져 역전된다.
- ▶ **실무 지침:** 소량·상호운용 → CSV, 대용량·분석 파이프라인 → Parquet. 형식 선택은 데이터 규모와 사용 패턴에 따라 달라져야 한다.

7 코드 분석 & 본인 의견

설계상 잘한 점

- ▶ **계층 분리:** 수집·검증·저장을 독립 모듈로 나눠 개별 테스트·교체가 가능. `run_pipeline()` 을 수집과 분리해 네트워크 없이도 검증·저장 로직을 테스트한다.
- ▶ **비동기 동시성:** `asyncio.gather()` 로 3개 API를 동시 호출해 순차 대비 총 대기시간을 최장 응답 하나 수준으로 단축. `return_exceptions=True` 로 일부 실패가 전체를 막지 않는다.
- ▶ **견고성:** 일시적 HTTP 오류(5xx·타임아웃)는 선형 백오프로 재시도하되, JSON 파싱 오류처럼 재시도가 무의미하면 즉시 실패 처리해 불필요한 지연을 피했다.
- ▶ **타입 안정성:** Pydantic v2로 타입뿐 아니라 범위(위경도 $\pm 90/\pm 180$, 강수량 0~100, 인구·면적 양수)까지 검증하고, 실패는 개별 예외 처리로 격리한다.

향후 개선 아이디어

- ▶ **테스트 커버리지:** `pytest-cov` 로 커버리지를 측정·리포트해 미검증 경로를 가시화(목표 80%+).
- ▶ **CI 자동화:** GitHub Actions에서 push 시 `ruff + pytest` 자동 실행으로 품질 게이트 구성(MLOps 연계).
- ▶ **대용량 확장:** 데이터가 커지면 Polars/청크 처리·Parquet 파티셔닝으로 메모리·속도 개선.
- ▶ **설정 외부화:** API URL·좌표·재시도 파라미터를 `.env` /설정 파일로 분리해 환경별 운용성 향상.

실습 소감

이번 실습을 통해 Parquet가 항상 CSV보다 효율적인 것은 아니며, 데이터 규모와 조회 방식에 따라 저장 형식을 선택해야 한다는 점을 실제 측정 결과로 확인했다. 또한 비동기 수집·스키마 검증·저장을 계층으로 분리하니 각 단계를 독립적으로 테스트하기 쉬웠고, 예외 처리와 재시도를 코드로 다뤄 보며 실무형 데이터 파이프라인의 기본 골격을 익힐 수 있었다.

검증 요약 — 3개 API 동시 수집 정상 · Pydantic 검증 통과 · CSV·Parquet 3종 저장·비교 · pytest 8건 통과 · ruff 오류 0 · Git 커밋 이력 다수.

🔗 PR #1 · Merged — github.com/kjung0109/skala-data/pull/1 | 코드 github.com/kjung0109/skala-data/tree/main/practice_day1